Day 132 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 19, Day 7 reference

Watch: Computer Vision - Lecture 11.3 (Self-Supervised Learning: Pretext Tasks) (Tübingen Machine Learning) | Self-Supervised Learning: Self-Prediction and Contrastive Learning, NeurIPS 2021 tutorial

`day132_self_supervised_build.py`, segment by segment, in the order `main()` uses things. The six exercise files' functions are pasted in verbatim (one banner per file); today adds the rotation task (rotate4, softmax_ce, train_rotation), the judges (probe_fit, knn_acc) and the demos. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: X = images, one per row · y = digits (used only to judge) · R = rotated images, r = their turns · reps = name → function from images to a representation

Segment 0 — Setup

Imports, and one line that makes the pasted parts work

CODE

```
HERE = os.path.dirname(os.path.abspath(__file__))
for w in ("week-18-unsupervised-learning-i", "week-17-classical-ml-kernels-ensembles"):
    sys.path.insert(0, os.path.join(HERE, "..", w))
with contextlib.redirect_stdout(io.StringIO()):
    import day123_exercise as d123
    import day119_exercise as d119
    import day117_exercise as d117
    import day116_exercise as d116
d126 = sys.modules[__name__]
ENC = ["relu", "linear"]; DEC = ["relu", "linear"]
SIZES_AE = [784, 256, 128, 256, 784]; ACTS_AE = ["relu", "linear", "relu", "linear"]
```

- `sys.path.insert(0, os.path.join(HERE, "..", w))` — earlier weeks' folders, so their exercise files can be imported.
- `d126 = sys.modules[__name__]` — the pasted parts of Days 127–131 call d126.forward and friends; here those functions live in this very file, so the name d126 is pointed at this module itself.
- `ENC = [...]; DEC = [...]` — Day 130's encode and decode read these names, so they are defined before the parts.

Segment 1 — The pretext task

rotate4 and softmax_ce

MATH

$p = softmax(L),\quad loss = -mean\ log\ p_{i,y_i},\quad \partial loss/\partial L = (p - onehot(y))\ /\ n$

CODE

```
def rotate4(X):
    I = X.reshape(-1, 28, 28)
    return np.vstack([np.rot90(I, k, axes=(1, 2)).reshape(len(X), -1) for k in range(4)]),
        np.repeat(np.arange(4), len(X))
def softmax_ce(L, y):
    m = L.max(1, keepdims=True); P = np.exp(L - m); P /= P.sum(1, keepdims=True); n = len(y)
    return float(-np.log(P[np.arange(n), y] + 1e-30).mean()), (P - np.eye(L.shape[1],
        dtype=L.dtype)[y]) / n
```

- `np.rot90(I, k, axes=(1, 2))` — turn every image a quarter turn k times, in the plane of axes 1 and 2 (rows and columns).
- `np.vstack([... for k in range(4)])` — all images turned 0 times, then all turned once, and so on: 4n rows.
- `np.repeat(np.arange(4), len(X))` — the matching labels: n zeros, n ones, n twos, n threes.
- `np.eye(4, dtype=L.dtype)[y]` — one-hot rows picked from an identity matrix, in the scores' precision.

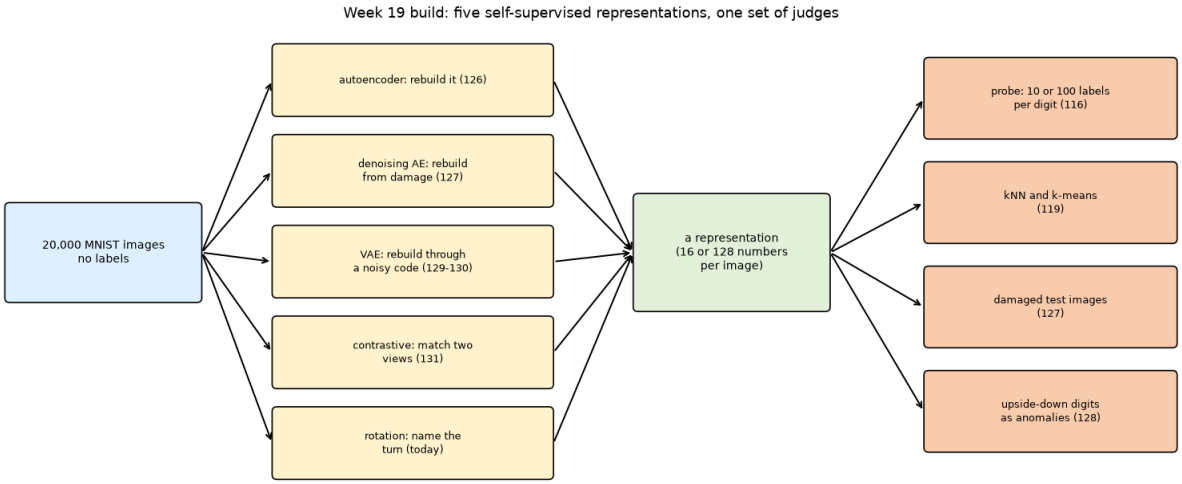
train_rotation

CODE

```
def train_rotation(X, sizes, epochs=10, batch=128, lr=1e-3, seed=0):
    acts = ["relu"] * (len(sizes) - 2) + ["linear"]; net = init_net(sizes, seed); opt = Adam(net,
        lr)
    R, r = rotate4(X); rng = np.random.RandomState(seed + 1); hist = []
    for _ in range(epochs):
        o = rng.permutation(len(R)); losses = []
        for s in range(0, len(R), batch):
            A = forward(net, R[o[s:s + batch]], acts); loss, d = softmax_ce(A[-1], r[o[s:s +
                batch]])
            opt.step(net, backward(net, A, acts, d)[0]); losses.append(loss)
        hist.append(float(np.mean(losses)))
    return net, hist
```

- `["relu"] * (len(sizes) - 2) + ["linear"]` — ReLU on every hidden layer, raw scores at the end.
- `R, r = rotate4(X)` — the whole training set, turned four ways, made once before training.
- `backward(net, A, acts, d)[0]` — Day 126's backward pass, used unchanged with a different loss gradient.

SYSTEM DIAGRAM



Five ways to learn without labels; the same judges for each.

Segment 2 — The judges

probe_fit: a classifier as a returned function

CODE

```
def probe_fit(R, y, n_per, seed=0):
    rng = np.random.RandomState(seed)
    tr = np.concatenate([rng.choice(np.where(y[:60000] == c)[0], n_per, replace=False) for c in
        range(10)])
    mu, sd = R[tr].mean(0), R[tr].std(0) + 1e-6
    W, b = d116.fit_softmax(((R[tr] - mu) / sd).astype(np.float64), y[tr], 10, lam=1e-2,
        steps=500)
    return lambda Z: (((Z - mu) / sd) @ W + b).argmax(1)
```

- `np.where(y[:60000] == c)[0]` — the training images of digit c (never the test images).
- `return lambda Z: ...` — a closure: the returned function remembers mu, sd, W and b, so it can be applied to clean or damaged images later.

train_all: every representation as a function

CODE

```
def train_all(X, rot_net):
    reps, secs = {}, {}
    mu, vals, V = d123.pca_fit(X[:20000].astype(np.float64))
    for k in (16, 128):
        reps[f"PCA {k}"] = lambda Z, k=k: d123.transform(Z.astype(np.float64), mu, V, k);
            secs[f"PCA {k}"] = None
    t0 = time.perf_counter(); ae, _ = train_ae(X[:20000], SIZES_AE, ACTS_AE, "bce", epochs=20,
        seed=0)
    secs["autoencoder 128"] = time.perf_counter() - t0; reps["autoencoder 128"] = lambda Z:
        forward(ae, Z, ACTS_AE)[2]
    t0 = time.perf_counter()
    dae, _ = train_ae(X[:20000], SIZES_AE, ACTS_AE, "bce", epochs=20, seed=0, noise=lambda xb,
        rng: masking(xb, 0.5, rng))
    secs["denoising AE 128"] = time.perf_counter() - t0; reps["denoising AE 128"] = lambda Z:
        forward(dae, Z, ACTS_AE)[2]
    t0 = time.perf_counter(); enc, dec, _ = train_vae(X[:20000], 16, epochs=30,
        Xval=X[60000:62000])
    secs["VAE means 16"] = time.perf_counter() - t0; reps["VAE means 16"] = lambda Z: encode(enc,
        Z)[0]
    t0 = time.perf_counter(); con, _ = train_contrastive(X[:20000], [784, 256, 128, 64], 0.1,
        epochs=30)
    secs["contrastive 128"] = time.perf_counter() - t0; reps["contrastive 128"] = lambda Z:
        represent(con, Z)
    reps["rotation 128"] = lambda Z: represent(rot_net, Z); secs["rotation 128"] = "sec. 2"
    reps["rotation, layer 1"] = lambda Z: forward(rot_net, Z, ["relu"])[1]; secs["rotation, layer
        1"] = "sec. 2"
    t0 = time.perf_counter(); rot1, _ = train_rotation(X[:20000], [784, 256, 128, 4], epochs=1)
    secs["rotation, 1 epoch"] = time.perf_counter() - t0; reps["rotation, 1 epoch"] = lambda Z:
        represent(rot1, Z)
    rnd = init_net([784, 256, 128, 4], 0); reps["untrained net 128"] = lambda Z: represent(rnd, Z)
    secs["untrained net 128"] = None
    return reps, secs, ae
```

- `lambda Z, k=k: d123.transform(...)` — k=k freezes the loop's current k; without it both PCA entries would use the last k.
- `forward(rot_net, Z, ["relu"])[1]` — forward zips layers with activations, so a one-item list runs only the first layer.
- `init_net([784, 256, 128, 4], 0)` — the same seed as train_rotation, so these are exactly the rotation network's starting weights.

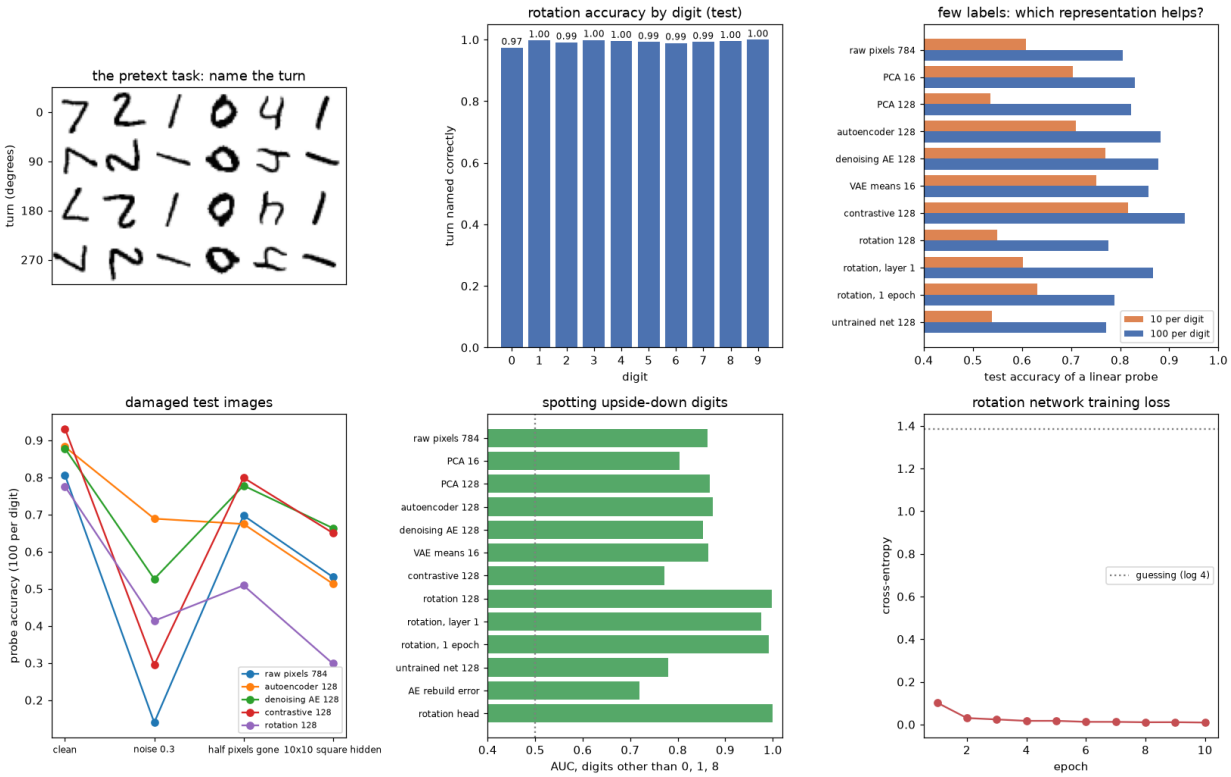
demo_upside_down: the rotation head as a detector

CODE

```
L = forward(rot_net, Q, ["relu", "relu", "linear"])[-1]; P = np.exp(L - L.max(1, keepdims=True));
    P /= P.sum(1, keepdims=True)
s = 1 - P[:, 0]; out["rotation head"] = (d117.roc_auc(lab, s), d117.roc_auc(lab[kk], s[kk]))
```

- `P[:, 0]` — the network's probability that the image was not turned.
- `1 - P[:, 0]` — a score that grows with the network's doubt that the image is upright; ROC AUC needs only its order.

REAL GRAPH



Real graph from the script.

Quick Syntax Glossary — new or reinforced constructs

- `sys.modules[__name__]` — the module object of the running file itself.
- `np.rot90(a, k, axes=(1, 2))` — turn arrays a quarter turn k times.
- `np.repeat(a, n)` — repeat each element n times.
- `return lambda Z: ...` — return a function that remembers its surroundings (a closure).
- `lambda Z, k=k: ...` — freeze a loop variable inside a lambda.
- `np.isin(a, values)` — True where a's element is one of the values.

End of Day 132 syntax reference. For the experiments and what they showed, see the Day 132 complete guide.